

MANUAL TÉCNICO

Mini ERP · Archivos

Ludvin Alexander Domingo Artola

Contenido

1. Introducción	3
2. Arquitectura	3
3. Tecnologías y requisitos	3
4. Estructura del proyecto	4
5. Base de datos	4
6. Seguridad y control de acceso	5
7. Reglas de negocio	6
7.1 Valuación por UEPS	6
7.2 Otras reglas	6
8. API REST	6
9. Reportes y factura	7
10. Cliente Angular	7
11. Instalación y puesta en marcha	7
12. Pruebas y mantenimiento	8

1. Introducción

Este manual describe cómo está construido el Mini ERP: su arquitectura, su base de datos, sus reglas de negocio y los pasos para instalarlo y mantenerlo. La forma de operar el sistema día a día se explica en el manual de usuario, que es un documento aparte.

El Mini ERP es un sistema de gestión comercial para una empresa que vende y distribuye muebles para hogar y oficina. Integra en una sola base de datos los productos y sus categorías, los proveedores, las compras, el inventario, los clientes y las ventas. Sobre esas operaciones ofrece control de acceso por área de trabajo, bitácora de lo que hacen los usuarios, doce reportes y la factura en PDF.

El inventario se valúa por UEPS (últimas entradas, primeras salidas), que es el método que corresponde al número de carné del estudiante por ser impar. El método es un parámetro de configuración, así que el sistema también funciona con PEPS.

2. Arquitectura

El sistema sigue una arquitectura cliente-servidor de tres capas físicas. Son dos aplicaciones independientes que se comunican por HTTP, más el manejador de base de datos.

La separación es estricta: el cliente no conoce la base de datos y el servidor es el único que abre conexiones a PostgreSQL y el único que aplica las reglas de negocio. Lo que valida el cliente es una comodidad para el usuario; el servidor vuelve a validar todo.

Dentro del servidor cada petición atraviesa siempre las mismas capas, y cada una tiene una sola responsabilidad:

Capa (paquete)	Responsabilidad
seguridad	Valida el token JWT y aplica la matriz de permisos antes de llegar al controlador.
controlador	Expone los endpoints REST y valida los datos de entrada. No contiene reglas de negocio.
servicio	Contiene las reglas de negocio y delimita las transacciones.
inventario	Motor de capas de costo (UEPS/PEPS), inventario y verificación de integridad.
repositorio, entidad	Acceso a datos con Spring Data JPA; las entidades corresponden una a una con las tablas.
dto, excepcion	Objetos de entrada y salida de la API y traducción uniforme de los errores.
auditoria, config, util	Bitácora automática, parámetros externos (IVA, valuación, empresa, CORS) y utilidades.

Por ejemplo, al registrar una venta el filtro valida el token, Spring Security comprueba el área, el controlador valida los datos, y el servicio abre una transacción en la que bloquea los productos, verifica existencias, descuenta las capas de costo, escribe el kardex, calcula los totales y registra la operación en la bitácora. Si algo falla, la transacción se revierte completa.

3. Tecnologías y requisitos

Componente	Tecnología y versión
Servidor	Java 21 · Spring Boot 4.1.1 (Web MVC, Data JPA, Security, Validation, AOP) · Maven 3.9+

Componente	Tecnología y versión
Base de datos	PostgreSQL 18 (funciona desde la 16) · migraciones con Flyway
Seguridad y PDF	JSON Web Token (jjwt 0.12.7) · OpenPDF 2.4.0
Cliente	Angular 21 con Angular Material · TypeScript 5.9 · Node.js 20.19+ / 22.12+ / 24+

4. Estructura del proyecto

MiniERP/	
MiniERPServer/	Servidor Spring Boot
pom.xml	Dependencias y construcción (Maven)
src/main/	
java/com/minierp/backend/	
auditoria/	Anotación @Auditable y aspecto de bitácora
config/	Parámetros de negocio, seguridad y empresa
controlador/	Endpoints REST
dto/ entidad/	Datos de la API y entidades JPA
excepcion/	Excepciones y manejador global de errores
factura/	PDF de la factura y número en letras
inventario/	Motor de capas UEPS e integridad
reporte/	Consultas SQL de los doce reportes
repositorio/	Repositorios Spring Data JPA
seguridad/	JWT, filtro y matriz de permisos
servicio/	Reglas de negocio por módulo
resources/	
application.properties	Configuración del servidor
db/migration/	Scripts de Flyway (V1 ... V4)
MiniERPClient/src/app/	Cliente Angular
nucleo/	API, sesión, interceptores, guardias, permisos
compartido/	Piezas reutilizadas (listados, formularios, gráficas)
diseno/	Marco de la aplicación y menú por área
paginas/	Una carpeta por módulo funcional
docs/	Documentación y diagramas
README.md	

5. Base de datos

La base de datos no se crea a mano ni la genera Hibernate: se construye con scripts SQL versionados que Flyway ejecuta al arrancar el servidor. Basta con crear la base vacía. Una migración ya aplicada nunca se modifica; todo cambio posterior va en un archivo nuevo. Además, Hibernate está configurado para solo validar el esquema, de modo que si una entidad no coincide con su tabla la aplicación avisa al arrancar en lugar de alterar la base.

Migración	Contenido
V1__esquema_inicial.sql	Tablas, llaves, restricciones, índices y vistas.
V2__datos_iniciales.sql	Carga inicial: roles, usuarios, 8 categorías, 30 productos, 8 proveedores, 20 clientes, 15 compras y 25 ventas con su inventario y su kardex.
V3__ajustes_y_numeracion.sql	Ajustes manuales de inventario y secuencias de numeración de documentos.
V4__bitacora_historica.sql	Bitácora de las operaciones de la carga inicial.

El esquema tiene dieciséis tablas:

Grupo	Tablas	Para qué sirven
Seguridad	rol, usuario	Las cuatro áreas de trabajo y sus usuarios.
Catálogos	categoria, producto, proveedor, cliente, proveedor_producto	Datos maestros. La última resuelve qué productos suministra cada proveedor y a qué costo.
Compras	compra, detalle_compra	Encabezado y líneas de cada compra.
Ventas	venta, detalle_venta	Encabezado y líneas de cada venta; la línea guarda el costo con que salió.
Inventario	capa_inventario, consumo_capa, movimiento_inventario	Capas de costo UEPS, su consumo y el kardex completo.
Auditoría	bitacora	Registro de todo lo que hacen los usuarios.

Hay además cuatro vistas de apoyo: vw_kardex (movimientos con su documento de origen), vw_existencia_calculada (existencia registrada frente a la del kardex), vw_valor_inventario (valor por producto según las capas con saldo) y vw_alerta_stock (productos en o por debajo de su mínimo).

La integridad no depende de que la aplicación se porte bien: el esquema declara llaves primarias y foráneas, restricciones UNIQUE en lo que no puede repetirse (código de producto, NIT, número de documento y de factura), restricciones CHECK en cantidades, costos y saldos, e índices para las consultas frecuentes. El modelo entidad-relación completo, en notación de Peter Chen, está en docs/diagramas/modelo-entidad-relacion.drawio.

6. Seguridad y control de acceso

El usuario ingresa con su usuario y contraseña en POST /api/auth/login y el servidor le devuelve un token JWT firmado con vigencia de ocho horas, junto con su área y sus permisos. A partir de ahí el cliente envía ese token en cada petición. Las contraseñas se guardan cifradas con BCrypt, el servidor no guarda sesiones (cada petición se valida por sí misma) y tanto el secreto de firma como la contraseña de la base de datos se leen de variables de entorno, nunca del código.

El sistema tiene cuatro áreas. Cada usuario pertenece a una sola y ve únicamente las opciones de su área; el menú del cliente se arma con los mismos permisos que aplica el servidor. «Total» significa consultar y registrar; «Consulta» es solo lectura.

Módulo	Administración	Compras	Inventario	Ventas
Categorías y productos	Consulta	Consulta	Total	Consulta
Proveedores y compras	Consulta	Total	Consulta	—
Cientes y ventas	Consulta	—	Consulta (ventas)	Total
Factura en PDF	Sí	—	—	Sí
Existencias y kardex	Consulta	Consulta	Total	—
Ajustes de inventario	—	—	Sí	—
Reportes	Todos	De compras	De inventario	De ventas
Bitácara	Sí	—	—	—

Administración observa todo el negocio y los reportes, pero no registra operaciones: así queda separado quien controla de quien opera. Cada operación relevante y cada ingreso al sistema quedan registrados en la tabla bitacora (quién, desde qué IP, qué módulo, qué acción, sobre qué registro y si tuvo éxito); lo escribe un aspecto asociado a la anotación @Auditable, y solo administración puede consultarla.

7. Reglas de negocio

7.1 Valuación por UEPS

Es la regla central del sistema. El inventario no se valúa con un costo promedio: se lleva por capas de costo.

- **Entrada.** Cada línea de compra crea una capa con su cantidad y su costo unitario; un ajuste de entrada también crea la suya.
- **Salida.** Cada línea de venta y cada ajuste de salida consumen capas empezando por la más reciente, que es lo que significa UEPS. Si una capa no alcanza, se agota y se continúa con la siguiente.
- **Costo de la salida.** Es el promedio ponderado de las porciones tomadas, y queda guardado en la línea: la utilidad de una venta no cambia, aunque después cambien los costos.
- **Trazabilidad.** Cada porción tomada se registra en consumo_capa, así que de cualquier venta se sabe de qué compras salió la mercadería.

7.2 Otras reglas

- Nunca se permite dejar existencias negativas; si la cantidad supera lo disponible, la operación se rechaza indicando cuánto hay.
- Durante una venta o un ajuste los productos se bloquean en la base de datos, para que dos operaciones simultáneas no consuman la misma capa.
- Cada movimiento del kardex guarda su origen (compra, venta o ajuste) con la existencia anterior y la nueva, de modo que el saldo se puede reconstruir.
- Los números de documento los asigna una secuencia de la base de datos: COM-2026-0001 para compras y FAC-2026-00001 para ventas.
- Los subtotales de las líneas los calcula la propia base como columnas generadas, y la tasa de IVA (12 %) queda congelada en la venta.
- Una compra solo puede incluir productos que el proveedor efectivamente suministre.
- Nada se borra: productos, categorías, clientes y proveedores se desactivan y conservan su historial. Las compras y ventas registradas no se modifican; cualquier corrección se hace con un ajuste de inventario.

8. API REST

Todas las rutas cuelgan de /api y viajan en JSON, con fechas en formato ISO-8601. Salvo el ingreso y la comprobación de estado, toda petición requiere la cabecera Authorization con el token. Los listados aceptan paginación y devuelven el total de elementos y de páginas. Los errores se devuelven siempre con el mismo

formato, con el código, un mensaje entendible y el detalle por campo en las validaciones, sin exponer la traza interna.

Los códigos de respuesta son los habituales: 200 y 201 en las operaciones correctas, 400 en datos inválidos, 401 sin token o con token vencido, 403 cuando el área no tiene permiso, 404 si el registro no existe y 409 cuando se incumple una regla de negocio, como existencia insuficiente o un código repetido.

9. Reportes y factura

Los doce reportes aceptan filtros y devuelven un encabezado con el título, la fecha, el usuario y los filtros aplicados, más un resumen con los totales. En el cliente se muestran con tabla, gráfica, exportación a CSV e impresión. Se resuelven con consultas SQL nativas de solo lectura en transacciones aisladas, para que el resumen y el detalle correspondan al mismo instante; el historial y la bitácora se devuelven paginados.

La factura se obtiene con GET /api/ventas/{id}/factura y la genera el servidor con OpenPDF: datos de la empresa y del cliente, número y fecha, detalle de líneas, subtotal, IVA, total y el total escrito en letras. Puede verse en el navegador o descargarse.

10. Cliente Angular

El cliente es una aplicación de página única con componentes standalone y signals, sin zone.js. Cada página se carga bajo demanda, de modo que el usuario solo descarga el código de los módulos que su área puede usar.

Carpeta	Contenido
nucleo	Servicio de API, sesión y token, interceptores de HTTP, guardias de ruta, mapa de permisos y manejador global de errores.
compartido	Piezas reutilizadas: listado base con filtros y paginación, formularios, buscador de productos, rango de fechas, gráficas y formatos.
diseno	Marco de la aplicación y construcción del menú según el área del usuario.
paginas	Una carpeta por módulo: login, inicio, categorías, productos, proveedores, clientes, compras, ventas, inventario y reportes.

Un interceptor agrega el token a cada petición y otro traduce los errores del servidor y cierra la sesión cuando el token deja de ser válido. Las guardias impiden entrar a una ruta sin sesión o sin el permiso que exige, usando el mismo mapa de permisos con que se arma el menú. Los doce reportes comparten una sola pantalla que se configura con una definición por reporte. Las fechas se muestran en dd/MM/aaaa, la moneda en quetzales, y las pantallas se exportan a CSV y se imprimen en horizontal.

11. Instalación y puesta en marcha

Requisitos: JDK 21, Maven 3.9 o superior, PostgreSQL 18 (o 16+) y Node.js 20.19+ / 22.12+ / 24+.

1. Crear la base vacía: CREATE DATABASE minierp; Las tablas y los datos los crea Flyway al arrancar.
2. Definir la contraseña de PostgreSQL como variable de entorno. En Windows: setx DB_PASSWORD "su_contraseña" (una sola vez; después hay que reabrir la terminal).

3. Arrancar el servidor: `cd MiniERPServer` y `mvn spring-boot:run`. Queda en `http://localhost:8080`.
4. Arrancar el cliente: `cd MiniERPClient`, `npm install` y `npm start`. Abrir `http://localhost:4200`.

Las demás variables de entorno tienen valores por omisión pensados para desarrollo: `JWT_SECRET` (secreto de firma del token), `DB_HOST`, `DB_PORT`, `DB_NAME` y `DB_USER` (ubicación y credenciales), `SERVER_PORT`, `CORS_ORIGINS` y el nivel de registro. En un equipo distinto del de desarrollo conviene definir al menos `DB_PASSWORD` y `JWT_SECRET`. Los parámetros de negocio (tasa de IVA, método de valuación y datos de la empresa que salen en la factura) están en `application.properties`, bajo el prefijo `minierp`.

Usuarios de la carga inicial, cuyas contraseñas son de demostración y deben cambiarse antes de usar el sistema con datos reales:

Usuario	Contraseña	Área
admin	admin123	Administración
jcompras	compras123	Compras
minventario	inventario123	Inventario
cventas	ventas123	Ventas

12. Pruebas y mantenimiento

El proyecto incluye pruebas unitarias del servidor (motor de capas UEPS y conversión de números a letras de la factura), pruebas unitarias del cliente (formatos, fechas y mapa de permisos) y guiones que recorren la API en funcionamiento comprobando catálogos, compras, ventas, inventario, reportes y factura contra lo esperado.

Además, `GET /api/inventario/verificacion` —y la pantalla de verificación del área de inventario— ejecuta diez comprobaciones sobre la base de datos: que la existencia de cada producto coincida con su kardex y con el saldo de sus capas, que cada salida haya consumido capas en el orden UEPS y con el costo correcto, que los totales de compras y ventas cuadren y que ningún producto tenga existencia negativa. Ninguna debe devolver fallos. Es la forma más rápida de confirmar que el inventario está sano después de una carga de datos o de una jornada de operaciones.

Para cambiar algo en la base de datos: escribir una migración nueva (V5, V6...) sin tocar las anteriores, actualizar la entidad JPA correspondiente y, si el dato debe verse por la API, ajustar el DTO, el servicio y el cliente. Las reglas de negocio se agregan en la clase de servicio del módulo, nunca en el controlador; los permisos se cambian en `ConfiguracionSeguridad.java` y en el mapa de permisos del cliente; la tasa de IVA, el método de valuación y los datos de la empresa se cambian en `application.properties`.

Antes de cualquier actualización conviene respaldar la base con `pg_dump`, y después de migrar datos, ejecutar la verificación de integridad.

Documentos relacionados: `README.md` (resumen y puesta en marcha rápida), `docs/diagramas/modelo-entidad-relacion.drawio` (modelo E-R en notación de Peter Chen), los diagramas UML y el manual de usuario, en la carpeta `docs`.